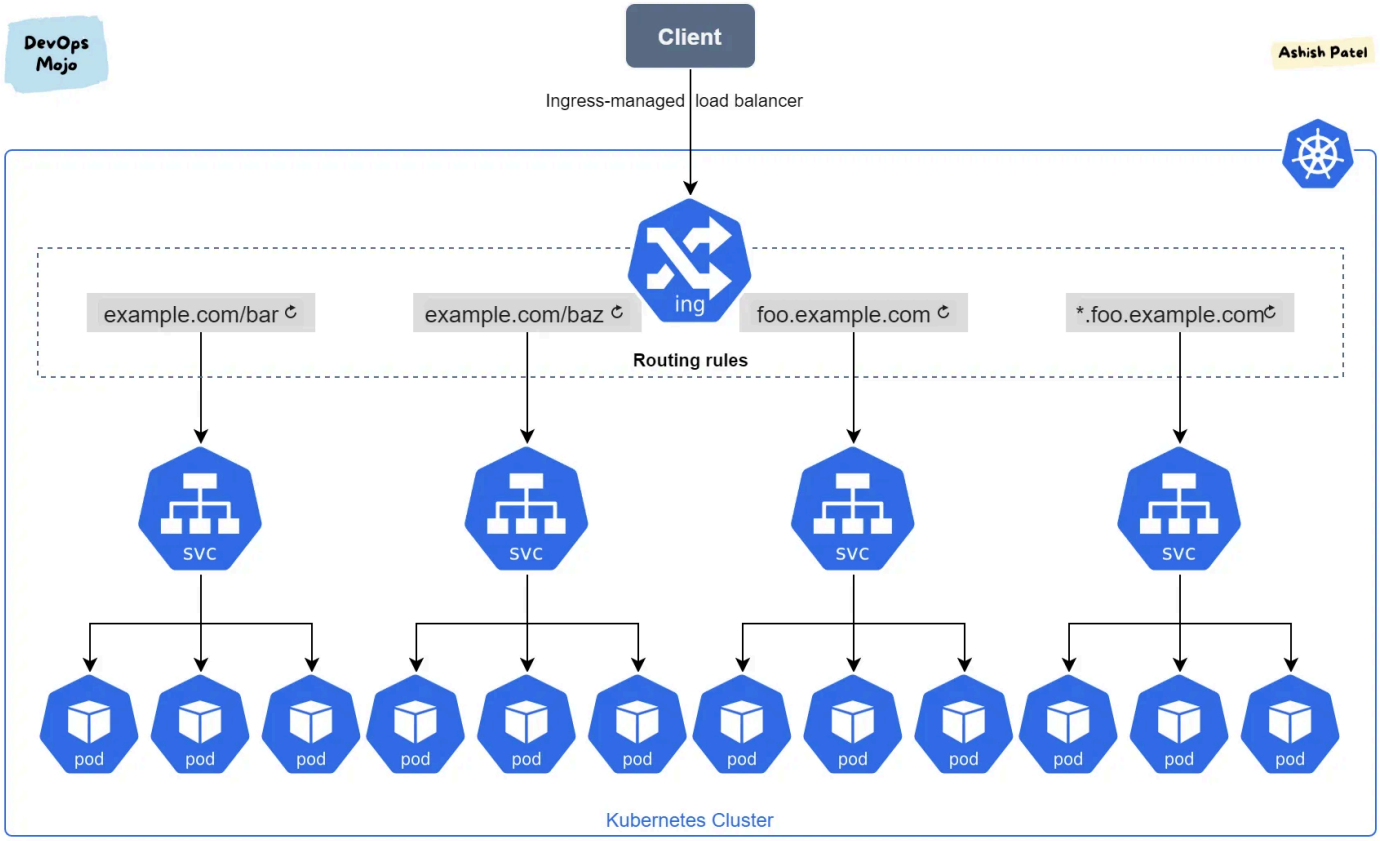
Kubernetes Ingress



What is Ingress?

Ingress is a Kubernetes API object used to manage and control access to services within a cluster. It acts as an intelligent router for HTTP and HTTPS traffic, allowing you to expose multiple services through a single IP address or LoadBalancer. Instead of exposing every service with a dedicated LoadBalancer or NodePort, you can configure an Ingress to direct traffic based on rules such as hostnames or URL paths.

This is especially useful in microservices architecture, where applications are broken down into multiple services that need to be accessed externally.

👉 **Example Use Case:** Imagine you're deploying a modern web application with the following components:

- A **Frontend Pod (UI)** that users interact with
- A **Backend Pod** that handles APIs and business logic
- A **Database Pod** used for internal storage and not meant to be accessed by users

With Ingress, you can:

- Route example.com/frontend to the frontend service
- Route example.com/backend to the backend service
- Keep the database secure and internal (not exposed to the outside world)

This approach simplifies external access and keeps sensitive components hidden.

Core Components of Ingress

Ingress consists of two critical parts that work together to handle traffic routing effectively:

1️⃣ Ingress Resources (Routing Rules)

These are Kubernetes API objects where you define the desired routing behavior for your cluster:

- Specify **hostnames**, such as example.com

- Configure **path-based rules**, such as `/api`, `/login`, `/dashboard`
- Map those paths to **target services** within the cluster

Ingress Resources are declarative and can be managed using YAML manifests. They tell the Ingress Controller how to route traffic, but they don't perform the routing themselves.

2️⃣ Ingress Controllers

The Ingress Controller is the actual component that reads the rules defined in Ingress Resources and implements the routing. It watches for changes in the Ingress Resources and dynamically configures the load balancing behavior.

Popular Ingress Controllers:

- **NGINX Ingress Controller**: Most widely used; supports annotations and advanced configurations
- **Traefik**: Lightweight, dynamic configuration; good for edge cases and developer setups
- **HAProxy**: High-performance, enterprise-grade load balancer
- **AWS ALB Ingress Controller**: Integrates directly with AWS Application Load Balancer; suitable for cloud-native setups on AWS

You need to deploy an Ingress Controller separately into your Kubernetes cluster. It is not built-in by default.

Why Use Ingress Instead of Load Balancer or NodePort?

Feature	Load Balancer	NodePort	Ingress
Cost Efficiency	Expensive (1 LB per service)	No cost (uses node's IP)	✅ Cost-effective (single entry point)
Custom Domains	Needs external DNS	Needs external DNS	✅ Supports custom domains (e.g., example.com)
Path-Based Routing	❌ No	❌ No	✅ Yes (`/frontend → frontend-service`)
SSL/TLS Termination	❌ No	❌ No	✅ Yes (handles HTTPS & TLS termination)

Key Benefits of Ingress

1. **Centralized Entry Point**
   - All incoming traffic can be managed through a single LoadBalancer and Ingress Controller.
   - Helps reduce the overhead of managing multiple entry points.
2. **Path-Based and Host-Based Routing**
   - Enables routing by URL path or domain name.
   - Great for multi-tenant applications or API versioning.
3. **SSL/TLS Termination**
   - Terminate HTTPS traffic at the Ingress level.
   - You can configure SSL certificates via Kubernetes secrets or external tools like cert-manager.
4. **Domain Name Flexibility**
   - Easily map subdomains (e.g., api.example.com, blog.example.com) to different services.
   - Ideal for separating different environments (dev, staging, prod).
5. **Scalability and Extensibility**
   - Supports advanced features via annotations (rate limiting, auth, rewrite rules).
   - Integrates with external authentication systems and monitoring tools.

🔐 How SSL/TLS Termination Works

Ingress Controllers can handle HTTPS traffic natively, making it easier to secure your applications without modifying the app itself:

1. **Client sends HTTPS request** to Ingress using a public SSL certificate.

2. **Ingress decrypts** the request using the configured certificate.
3. **Ingress forwards** the decrypted HTTP request to the backend services within the cluster.

This means your internal services only need to support HTTP, simplifying service configuration while maintaining external security. Tools like **cert-manager** can help automate certificate provisioning and renewal.

Why Use Ingress?

Feature	Benefit
✅ Centralized Routing	One entry point for multiple services.
✅ Custom Domains	Supports DNS-based routing (example.com).
✅ Cost-Effective	Reduces the need for multiple Load Balancers.
✅ Path-Based Routing	Routes `/frontend` to Frontend Service, `/backend` to Backend Service.
✅ SSL/TLS Termination	Manages HTTPS termination for security.

Conclusion

- Ingress is essential for managing external HTTP/S traffic in Kubernetes.
- It allows cost-efficient, scalable, and secure service exposure.
- It eliminates the need for multiple Load Balancers, reducing complexity.

Setting Up Nginx Ingress and SSL in Kubernetes

Overview

This guide explains how to configure Nginx Ingress in a Kubernetes cluster and set up SSL/TLS certificates using Let's Encrypt and Cert-Manager.

1️⃣ Installing Nginx Ingress Controller

Why Install Nginx Ingress Controller?

- Manages external access to applications inside the Kubernetes cluster.
- Routes requests based on rules (URLs, services, etc.).
- Acts as a reverse proxy and load balancer.

Steps to Install Nginx Ingress

1. Apply the Nginx Ingress YAML file "kubectl apply -f <nginx-ingress.yaml>"
   - This deploys the Nginx Ingress controller and sets up required permissions.
2. Verify Nginx Ingress Controller Pods "kubectl get pods -n ingress-nginx"
   - Ensure the pods are in Running state.

2️⃣ Installing Cert-Manager

Why Cert-Manager?

- Automates SSL/TLS certificate management.
- Integrates with Let's Encrypt for issuing certificates.
- Manages certificate renewals automatically.

Steps to Install Cert-Manager

1. Apply the Cert-Manager YAML file "kubectl apply -f <cert-manager.yaml>"

- This sets up RBAC roles, service accounts, and other necessary resources.
2. Verify Cert-Manager Installation "kubectl get pods -n cert-manager"
   - Ensure that all Cert-Manager pods are running.

3️⃣ Setting Up Let's Encrypt ClusterIssuer

Why ClusterIssuer?

- Defines the certificate issuing authority for the cluster.
- Specifies Let's Encrypt as the certificate provider.

Steps to Create ClusterIssuer

1. Create a ClusterIssuer YAML file (`ci.yaml`):
2. Apply the ClusterIssuer configuration
   - Replace devops@example.com with your email.
3. Verify ClusterIssuer Creation

4️⃣ Configuring DNS for Your Domain

Why Configure DNS?

- To map your domain name (e.g., example.com) to your Kubernetes Ingress LoadBalancer.
- This allows users to access the application using a friendly domain name.

Steps to Configure DNS

1. Find the External LoadBalancer IP/URL
   - Copy the External IP or URL from the output.
2. Update DNS Records
   - If you have an IP address (e.g., AKS, GKE, EKS):
     - Add an A record:
   - If you have a LoadBalancer URL:
     - Add a CNAME record:
   - Use dnschecker.org to verify propagation.

Nginx Ingress & SSL Setup for Production

Step 0: Deploy Your Application

Ensure your Kubernetes application (e.g., microservices, APIs) is already deployed and running before setting up Ingress and SSL. This setup assumes services are accessible via internal Kubernetes Service objects.

1. Install Nginx Ingress Controller (Optimized)

Instead of using the default manifest, install the Nginx Ingress Controller using Helm.Why?

- Helm allows easier upgrades and better control over configurations.
- You can customize parameters like load balancer type, logging, and security settings.

✅ Install Helm (if not already installed)

```
curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
```

✅ Add the Nginx Ingress Controller Helm repo

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update
```

✅ Install Nginx Ingress Controller with Production Settings

```
helm install nginx-ingress ingress-nginx/ingress-nginx \
  --namespace ingress-nginx --create-namespace \
  --set controller.replicaCount=2 \
  --set controller.service.type=LoadBalancer \
  --set controller.service.externalTrafficPolicy=Local \
  --set controller.admissionWebhooks.enabled=false \
  --set controller.metrics.enabled=true \
  --set controller.config.enable-modsecurity=true
```

✅ Optimizations:

- `replicaCount=2` → Ensures high availability by running two replicas.
- `externalTrafficPolicy=Local` → Preserves the client's IP address (useful for logging & security).
- `admissionWebhooks.enabled=false` → Prevents potential issues with webhook dependencies.
- `metrics.enabled=true` → Enables Prometheus metrics for monitoring.
- `enable-modsecurity=true` → Enables security module (adds protection against attacks).

✅ Verify that the Ingress Controller is running

```
kubectl get pods -n ingress-nginx
```

2. Install Cert-Manager (Optimized)

Instead of applying YAML manifests manually, **use Helm for Cert-Manager installation**.

✅ Add the Cert-Manager Helm repository

```
helm repo add jetstack https://charts.jetstack.io
helm repo update
```

✅ Install Cert-Manager with Production Settings

```
helm install cert-manager jetstack/cert-manager \
  --namespace cert-manager --create-namespace \
  --set installCRDs=true
```

✅ Why Helm?

- It automatically manages CRDs (Custom Resource Definitions).
- Easier to upgrade Cert-Manager in the future.
- More secure and stable than raw YAML installations.

✅ Verify Cert-Manager is Running

```
kubectl get pods -n cert-manager
```

Configure Let's Encrypt ClusterIssuer (Optimized)

- Current Issue: Jumps straight to production issuer—risks hitting Let's Encrypt rate limits during testing. Missing optimization for solvers.
- Optimized Approach:
  - Add a staging issuer first to test, then switch to production.

- **Why**:
  - **Staging**: Avoids Let's Encrypt production rate limits (50 certs/week/domain) during setup.
  - **Solver Template**: Ensures consistent Ingress annotations, reducing misconfigs.
- **Reliability**: Staging catches issues before production certs are requested.

✅ Create the ClusterIssuer YAML

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    OR
    server: https://acme-staging-v02.api.letsencrypt.org/directory
    email: devopsbyaditya@gmail.com  # Replace with your email
    privateKeySecretRef:
      name: letsencrypt-prod
    solvers:
    - http01:
        ingress:
          class: nginx
```

✅ Apply the ClusterIssuer

```
kubectl apply -f letsencrypt-clusterissuer.yaml
```

✅ Optimizations:

- Uses ACME HTTP-01 challenge (more stable and works with most DNS providers).
- No unnecessary settings; minimalistic and production-ready.

What Happens When Cert-Manager Calls This URL?

1. Cert-Manager sends a request to this URL with your domain name.
2. Let's Encrypt validates your domain using the HTTP-01 challenge.
3. If validation is successful, Let's Encrypt issues an SSL certificate.
4. Cert-Manager stores the certificate in a Kubernetes Secret.
5. Your Ingress automatically uses the certificate for HTTPS.

4. Create Ingress Resource (Optimized)

✅ Save the following YAML as `ingress.yaml`

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: bankapp-ingress
  namespace: default
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
    nginx.ingress.kubernetes.io/force-ssl-redirect: "true"
```

```
    nginx.ingress.kubernetes.io/rewrite-target: /
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
    nginx.ingress.kubernetes.io/proxy-body-size: "50m"  # Prevents file upload failures
    nginx.ingress.kubernetes.io/server-snippet: |
      location /healthz {
        access_log off;
        return 200;
      }
spec:
  ingressClassName: nginx
  rules:
  - host: www.shackverse.co
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: bankapp-service
            port:
              number: 80
  tls:
  - hosts:
    - www.shackverse.co
    secretName: shackverse-co-tls
```

✅ Apply the Ingress Resource

```
kubectl apply -f ingress.yaml
```

✅ Optimizations:

- proxy-body-size: "50m" → Prevents issues with large file uploads.
- server-snippet → Defines a health check endpoint (`/healthz`) for monitoring.

5. Add Load Balancer URL to DNS

Once the Ingress is created, find the LoadBalancer URL and update your DNS settings.

✅ Get Load Balancer URL

```
kubectl get svc -n ingress-nginx
```

Look for the EXTERNAL-IP of the nginx-ingress-controller and update your DNS records.

✅ Optimizations:

- Use Route 53 (AWS) or Cloudflare for faster propagation.
- Enable CDN caching for faster global access.

6. Verify the Setup

✅ Check SSL Certificate Status

```
kubectl get certificates
```

✅ Look for `Ready: True` .

✅ Verify Ingress Configuration

```
```

```
kubectl describe ingress bankapp-ingress
```

✅ Should show **TLS enabled** and traffic routing correctly.

✅ Test HTTPS Access

```
curl -v https://www.shackverse.co
```

✅ Should return **200 OK** with valid SSL.

✅ Check Cert-Manager Logs (If Issues Occur)

```
kubectl logs -n cert-manager -l app=cert-manager
```

Final Steps Summary

Step	Optimization
1. Install Nginx Ingress Controller	Use Helm instead of raw YAML for better control & HA
2. Install Cert-Manager	Use Helm to manage CRDs & automatic updates
3. Create Let's Encrypt ClusterIssuer	Minimalistic ACME HTTP-01 challenge
4. Create Ingress Resource	Added upload limit, health check, security annotations
5. Update DNS with Load Balancer	Get LB IP and add DNS records
6. Verify SSL & Ingress	Run checks for certificates, ingress, logs, and HTTPS connectivity

Roles and responsibilities

Component	Role in Kubernetes	What It Does? (Detailed Explanation)
Cert-Manager	Manages SSL certificates in Kubernetes	- Automates the process of requesting, issuing, renewing, and storing SSL/TLS certificates. - Ensures that certificates are automatically renewed before expiration. - Stores issued certificates in Kubernetes Secrets so Ingress resources can use them for HTTPS.
ClusterIssuer	Acts as a certificate request manager	- Defines where and how to request SSL/TLS certificates. - Contacts a Certificate Authority (CA) like Let’s Encrypt to get a certificate. - Ensures that all namespaces in the cluster can request certificates using a single issuer.
Let’s Encrypt	Acts as the Certificate Authority (CA)	- Issues SSL/TLS certificates to verify that a domain is secure. - Provides a public ACME API endpoint for requesting certificates. - Ensures that certificates follow security and encryption standards. - Requires domain validation before issuing certificates (via HTTP-01 or DNS-01 challenges).

server: https://acme-v02.api.letsencrypt.org/directory	ACME API endpoint for certificate requests	- This is the official Let's Encrypt ACME API URL used to request certificates. - Cert-Manager sends certificate requests to this server. - Let's Encrypt validates the domain ownership and then issues a certificate.
Ingress TLS Annotation (cert-manager.io/cluster-issuer)	Enables HTTPS for your domain	- This annotation in the Ingress resource tells Cert-Manager to request a certificate for the domain. - Ensures that the application is served over HTTPS. - Triggers Cert-Manager to store the certificate in a Kubernetes Secret.
Secret (`shackverse-co-tls`)	Stores issued SSL/TLS certificate	- Cert-Manager creates this Kubernetes Secret after obtaining an SSL certificate from Let's Encrypt. - The Ingress controller uses this Secret to enable HTTPS for the application. - Contains both the TLS certificate and private key, securely stored inside the cluster.

Detailed Explanation of How Everything Works Together in Production

1️⃣ Cert-Manager is installed and manages all SSL/TLS certificates in the cluster.
2️⃣ ClusterIssuer is configured to tell Cert-Manager to request certificates from Let's Encrypt.
3️⃣ Let's Encrypt (CA) verifies that the domain is valid and issues a TLS certificate.
4️⃣ Cert-Manager contacts Let's Encrypt's ACME API server at https://acme-v02.api.letsencrypt.org/directory to request the certificate.
5️⃣ Cert-Manager stores the certificate in a Kubernetes Secret (`shackverse-co-tls`).
6️⃣ The Ingress resource references the Secret and enables HTTPS for www.shackverse.co.
7️⃣ Traffic now securely reaches the application over HTTPS using a valid SSL/TLS certificate.
8️⃣ Cert-Manager automatically renews the certificate before expiration, ensuring continuous HTTPS without manual intervention.

7️⃣ Troubleshooting Tips

Issue	Solution
LoadBalancer IP/URL not assigned	Wait for a few minutes and run `kubectl get ingress` again.
SSL certificate not issued	Check `kubectl describe certificate tls-secret` for errors.
DNS not propagating	Use dnschecker.org to verify, or wait for 1-2 hours.
Application not accessible	Ensure your Ingress, Service, and Deployment are properly configured.

✅ Conclusion

By following these steps, you have:

1. Installed Nginx Ingress to manage external traffic.
2. Set up Cert-Manager for automated SSL certificates.
3. Configured Let's Encrypt ClusterIssuer for secure HTTPS connections.
4. Mapped a custom domain to your application.
5. Deployed a secure Kubernetes Ingress for external access.

Configuring Domain and DNS Records for Kubernetes Load Balancer

1️⃣ Prerequisites

Before proceeding, ensure that:

1. You own a domain (e.g., stackverse.com).
2. You have admin access to the DNS provider where you registered your domain.
3. Your Kubernetes cluster has a LoadBalancer service running.

2️⃣ Understanding DNS Records

There are two main DNS record types you will work with when connecting a domain to your Kubernetes LoadBalancer: **A Records** and **CNAME Records**.

✅ A Record (Address Record)

What it does: Maps a domain or subdomain to a direct IP address.
Use Case Example: You deployed your app on a Kubernetes cluster(AKS), and your LoadBalancer exposes an **external IP** like 34.95.100.25.
To map your domain to this IP:

- Go to your domain registrar's DNS settings.
- Add an **A Record**:
  - Name: `@`
  - Type: `A`
  - Value: 34.95.100.25

This means stackverse.com will now point to your app hosted on that LoadBalancer IP.
You can also add:

- Name: `www`
- Type: `A`
- Value: 34.95.100.25

So that www.stackverse.com also resolves correctly.

✅ CNAME Record (Canonical Name Record)

What it does: Maps a **subdomain** to another **domain name** (not to an IP directly).
Use Case Example: Some cloud providers (like AWS or GCP) do not expose a fixed IP but instead provide a LoadBalancer **URL** like a1b2c3d4e5.cloudprovider.net.
To map www.stackverse.com to this LoadBalancer:

- Add a **CNAME Record**:
  - Name: `www`
  - Type: `CNAME`
  - Value: a1b2c3d4e5.cloudprovider.net

⚠️ Important Note: CNAME **cannot be used on the root domain** (`@` or stackverse.com). It is allowed only on subdomains like `www`, `app`, `blog`, etc.

Choosing Between A Record and CNAME

Scenario	Use
LoadBalancer exposes an IP Address	Use A Record
LoadBalancer provides a URL	Use CNAME Record

Configuring DNS Records

For LoadBalancers with IP Addresses (A Record)

1. **Get the LoadBalancer External IP**

Example Output:

- The **external IP address** is **192.168.1.10**.

1. **Go to your Domain Provider's DNS Settings**Add an **A Record**:
   - **@** → Maps to stackverse.com
   - **www** → Maps www.stackverse.com to 192.168.1.10 (if needed)

```
kubectl get ingress
NAME        CLASS    HOSTS            ADDRESS      PORTS
my-ingress  nginx    stackverse.com    192.168.1.10  80, 443
Name: @
Type: A
Value: 192.168.1.10
```

For LoadBalancers with URLs (CNAME Record)

1. Get the LoadBalancer External URL

Example Output:

- The LoadBalancer URL is abc123.cloudprovider.com.

1. Go to your Domain Provider's DNS SettingsAdd a CNAME Record:
   Important Note:
   - You cannot use `@` with CNAME.
   - CNAME only works with subdomains like `www`.

```
kubectl get ingress
NAME        CLASS    HOSTS            ADDRESS
my-ingress  nginx    stackverse.com    abc123.cloudprovider.com
Name: www
Type: CNAME
Value: abc123.cloudprovider.com
```

Verifying DNS Propagation

Once the records are updated, DNS changes may take time (5 minutes to 24 hours) to reflect globally.

Using Online DNS Checker

1. Visit: https://dnschecker.org
2. Enter your domain: www.stackverse.com
3. Select record type: `CNAME` or `A`
4. Click "Search"
   - Ensure the resolved IP/URL matches your LoadBalancer.

Using `nslookup` in Terminal

1. Check DNS ResolutionExpected Output:
   - If using CNAME, it should resolve to the LoadBalancer URL.

```
nslookup www.stackverse.com
Server:  8.8.8.8
```

```
Address: 8.8.8.8#53
Non-authoritative answer:
Name: www.stackverse.com
Address: 192.168.1.10
```